

Multi-Agent Database Analyzer

Architecture, System Design and Agentic Chat Support

Sujeet Banerjee

Abstract

This paper presents the reference architecture and system design of a multi-agent Database Analyzer that enables users to ask natural-language questions across large-scale enterprise databases and receive evidence-based summaries, root-cause analyses, and conversational insights. The core architectural design principle is that **the LLM acts as the planner and interpreter, while the underlying data platform performs heavy computation and serves as the factual evidence base**. The framework incorporates metadata RAG for schema discovery, semantic metric cataloging, self-healing Text-to-SQL generation, deterministic statistical analysis, and strict out-of-band security guardrails.

Contents

1	Problem Statement	2
2	System Goals	2
3	Reference Architecture	3
4	Separation of Agent Responsibilities	3
5	Agentic Investigation Loop & Self-Healing	5
6	Schema RAG and Semantic Layer	5
7	Safe Text-to-SQL Execution Path	6
8	Large-Scale Data Flow Compute Pushdown	6
9	Evidence Binding and Lineage	7
10	Defense-in-Depth Security Model	7
11	System Bottlenecks Mitigations	7
12	Recommended Proof-of-Concept (POC) Roadmap	7
13	Key Architectural Takeaway	8

1 Problem Statement

Consider an enterprise analytical environment with databases containing billions of rows covering customers, transactions, orders, support interactions, and operational metrics.

When a user asks:

“Summarize our business performance last quarter. Why did revenue decline, and which products and customer segments caused the decline?”

A naive LLM chatbot cannot ingest raw database tables due to context length limits, extreme token cost, latency, and security risks. Conversely, fixed Text-to-SQL scripts lack multi-step reasoning capabilities to follow up on unexpected anomalies.

An enterprise-grade solution requires an agentic system that acts like a senior data analyst:

1. Understands user intent and disambiguates business metrics.
2. Dynamically discovers relevant schema subsets using Metadata RAG.
3. Formulates an analytical execution plan.
4. Generates safe, highly optimized SQL and auto-corrects execution errors.
5. Pushes heavy aggregations down to the database/OLAP engine.
6. Applies deterministic statistical checks on returned evidence.
7. Decides whether additional investigative drill-downs are required.
8. Synthesizes evidence into clear, auditable explanations with query lineage.

Database $\rightarrow$ LLM $\rightarrow$ Summary

 (Infeasible for Enterprise Scale)

User $\rightarrow$ Planner $\rightarrow$ Metadata / Semantics $\rightarrow$ Safe SQL $\rightarrow$ Database $\rightarrow$ Analysis $\rightarrow$ Evidence $\rightarrow$ LLM Summary

2 System Goals

- **Natural Language Access:** Seamless chat-based exploration of structured enterprise data.
- **Decoupled Heavy Computation:** Execution remains close to the data platform (Snowflake, BigQuery, Databricks, Postgres).
- **Semantic Precision:** Metric definitions (e.g., Net Revenue) are resolved via a centralized catalog.
- **Bounded Autonomy:** Agentic investigation loops operate under strict budget, latency, and step limits.
- **Defense-in-Depth Security:** RBAC, row-level access control, and PII masking are enforced by the database, not by prompt instructions.
- **Full Traceability:** Every claim in a summary is linked to execution logs, SQL queries, and source tables.

3 Reference Architecture

Figure 1 illustrates the layered reference architecture of the Multi-Agent Database Analyzer.

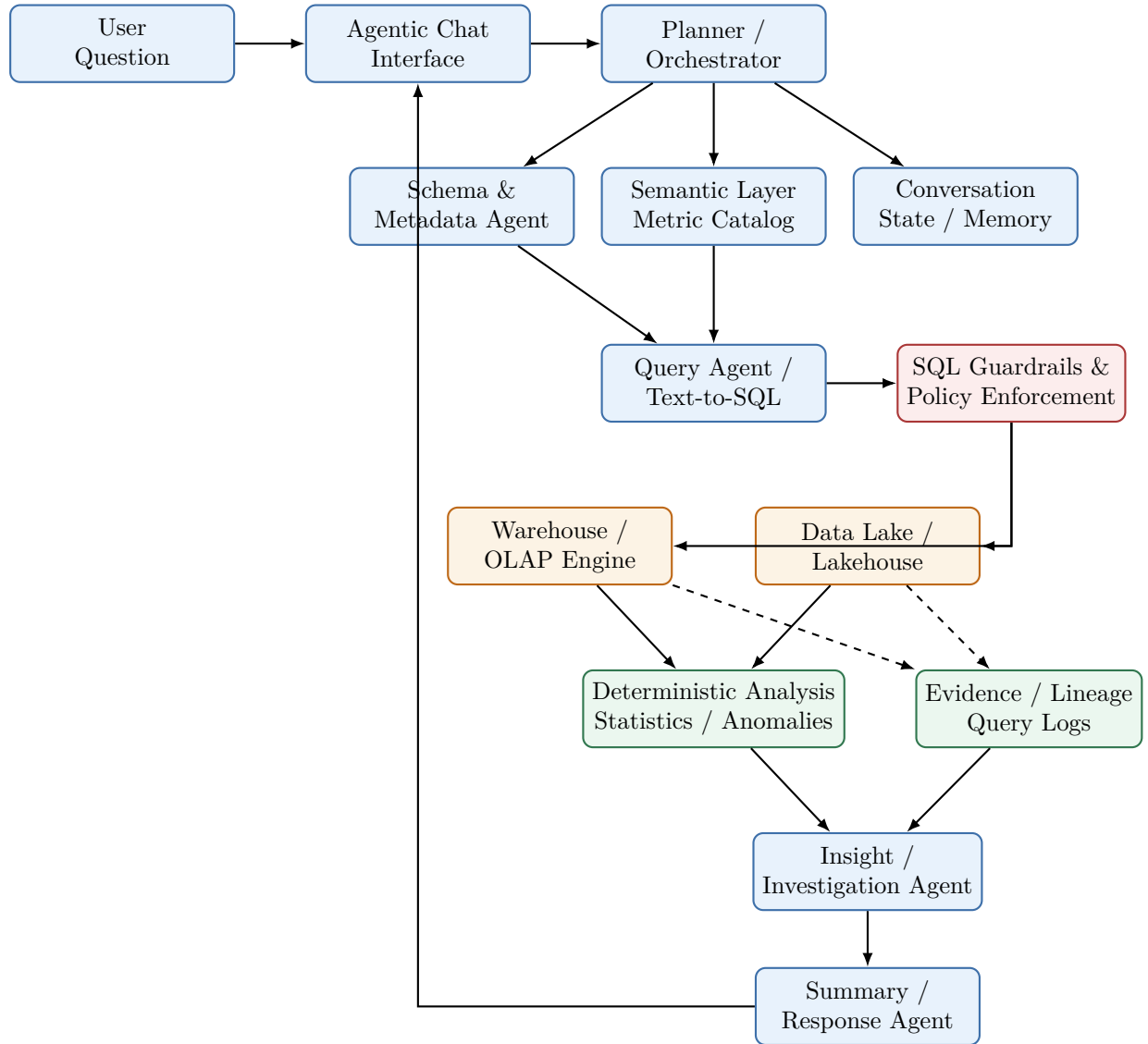


Figure 1: Layered reference architecture for multi-agent database analysis.

Core Architectural Principle: The LLM plans and interprets; the data platform performs large-scale computation and provides verifiable evidence.

4 Separation of Agent Responsibilities

Rather than using a single monolithic prompt, responsibilities are partitioned into specialized functional agents or modular sub-routines.

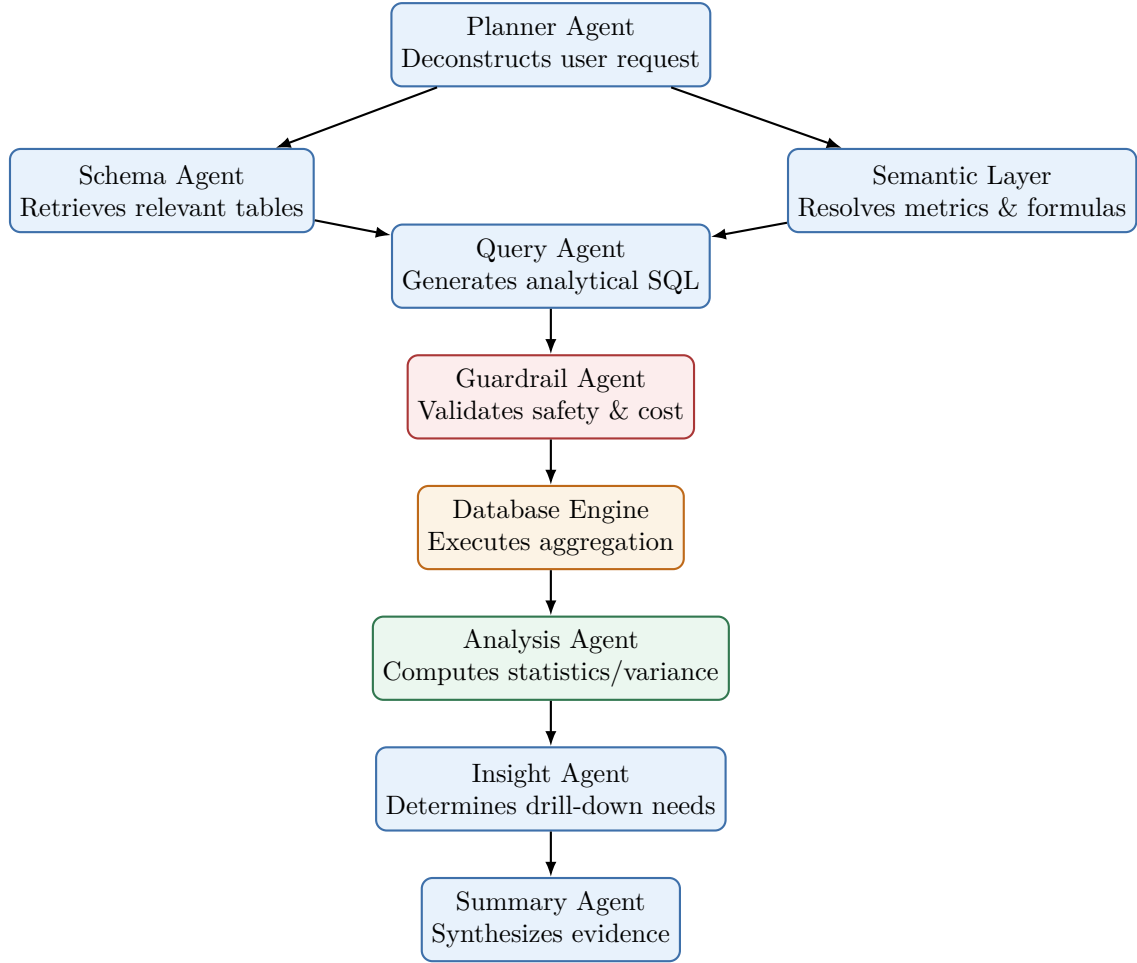


Figure 2: Modular agent separation of concerns.

Agent / Module	Primary Responsibility
Planner	Converts multi-part user prompts into structured analytical plans.
Schema Agent	Performs hybrid retrieval over metadata catalogs to extract minimal schema details.
Semantic Layer	Maps business terminology (e.g., “churn”) to certified database columns and logic.
Query Agent	Generates dialect-specific analytical SQL with CTEs and window functions.
Guardrail Agent	Inspects ASTs for safety, syntax correctness, cost ceilings, and RBAC rules.
Analysis Agent	Executes deterministic statistical tests (variance, trend, anomalies) on aggregate results.
Insight Agent	Evaluates whether findings satisfy user intent or require further drill-down.
Summary Agent	Translates verified evidence sets into human-readable narratives with citations.
Memory State	Maintains conversation history, filter bindings, and entity references.

5 Agentic Investigation Loop & Self-Healing

An investigation loop allows the system to drill down autonomously when initial aggregates indicate anomalies (e.g., detecting a regional revenue drop and automatically querying top-affected products).

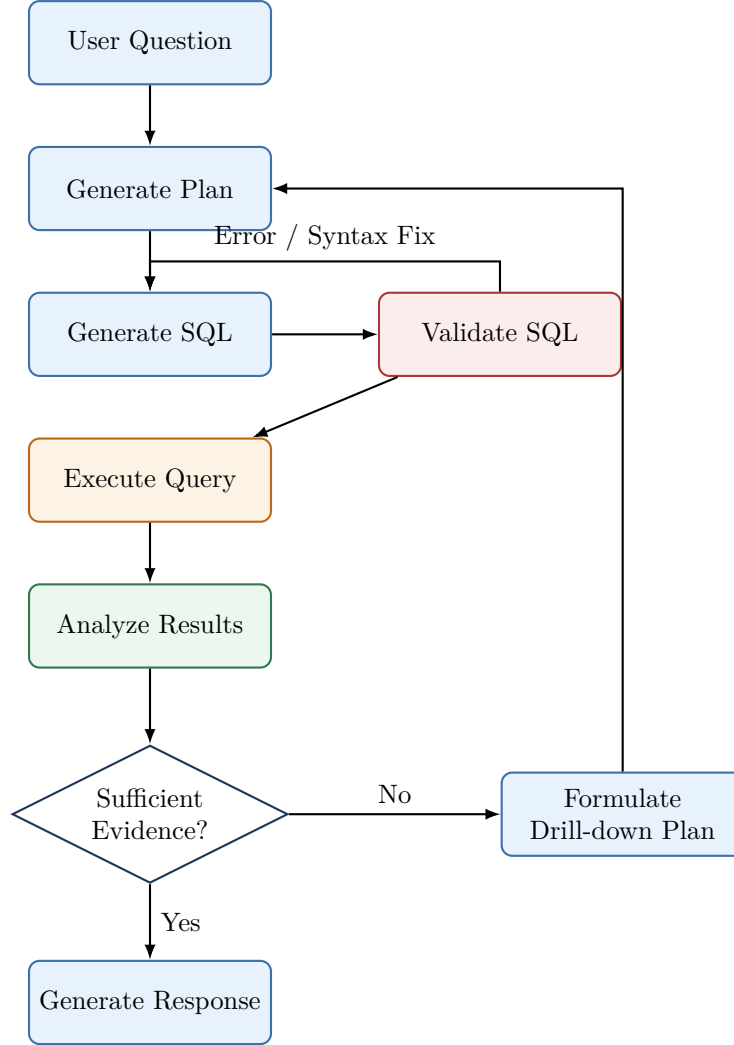


Figure 3: Bounded agentic investigation loop with SQL self-healing feedback.

To prevent runaway costs or infinite execution loops, investigation loops must be governed by hard boundaries:

- **Max Iteration Depth:** Limit to a maximum of 3–4 investigation steps per request.
- **Query Budget Ceiling:** Restrict maximum execution time and scanned data quotas per session.
- **Result Set Cap:** Truncate statistical output before sending back to the LLM context.

6 Schema RAG and Semantic Layer

Supplying an entire corporate database DDL (thousands of tables) to an LLM induces hallucinations and context degradation. The architecture implements **Metadata RAG** combined with a **Semantic Layer**.

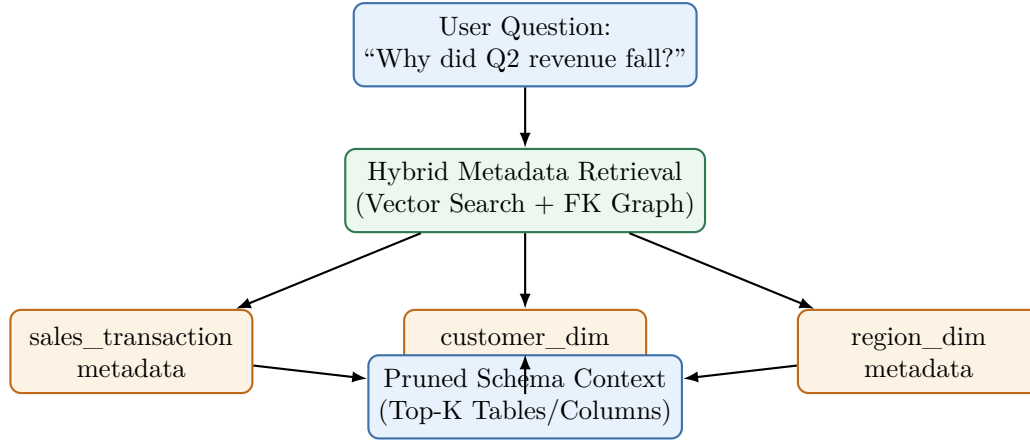


Figure 4: Metadata RAG for dynamically indexing database schemas.

The **Semantic Layer** standardizes complex business logic into verifiable metrics, ensuring that terms like “Net Revenue” are calculated consistently.

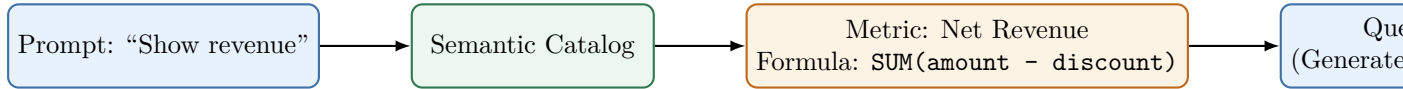


Figure 5: Semantic disambiguation preceding query generation.

7 Safe Text-to-SQL Execution Path

To guarantee database safety, query generation must pass through strict validation gates before hitting production engines.

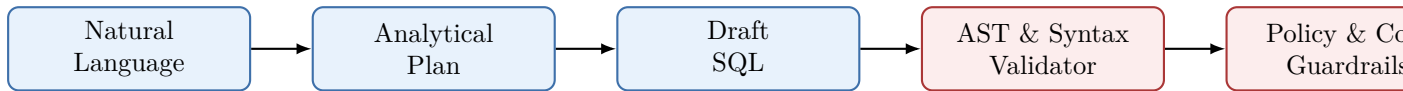


Figure 6: Safe execution pipeline for generated queries.

8 Large-Scale Data Flow Compute Pushdown

The architecture guarantees scalability by executing all heavy transformations, joins, and aggregations directly within the database engine.

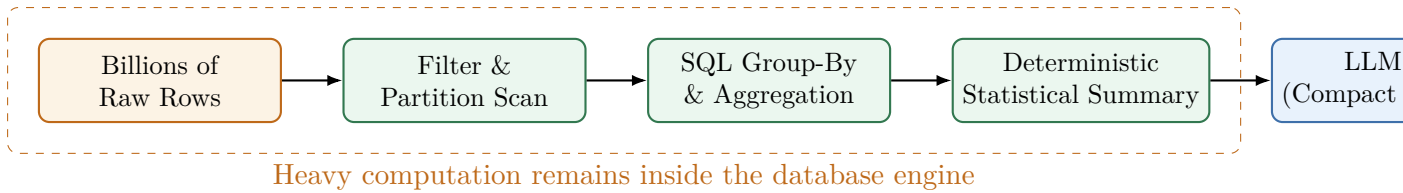


Figure 7: Compute pushdown pipeline reducing massive raw data to small evidence sets.

9 Evidence Binding and Lineage

Every insight presented in the conversational output is bound to specific execution traces. This ensures that conclusions are fully verifiable and auditable.

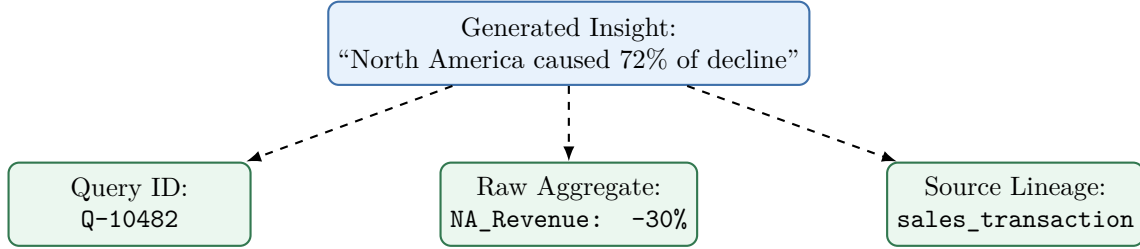


Figure 8: Evidence binding architecture for auditability and lineage.

10 Defense-in-Depth Security Model

Security controls must be enforced outside the LLM. The model should never be trusted as an access control mechanism.

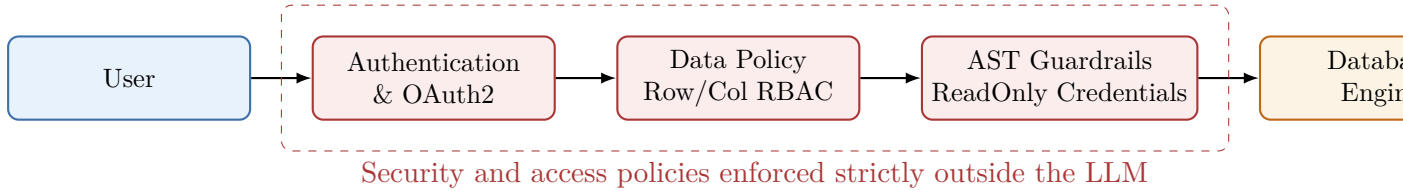


Figure 9: Out-of-band security and access enforcement model.

11 System Bottlenecks Mitigations

Bottleneck	Risk / Impact	Architectural Mitigation
Context Overflow	Massive DDLs exceed LLM context window.	Metadata RAG and top-K column indexing.
Expensive Scans	Poorly constructed queries lock database tables.	Query cost estimators, timeouts, and required partitions.
Metric Confusion	LLM hallucinating metric formulas.	Centralized Semantic Layer / Metric Catalog.
Loop Explosion	Infinite agentic investigation loops.	Bounded iteration steps and query budget limits.
Security Violations	Prompt injection attempting data exfiltration.	Out-of-band database RBAC and read-only connections.

12 Recommended Proof-of-Concept (POC) Roadmap

A phased roadmap for implementing a production-grade database analyzer:

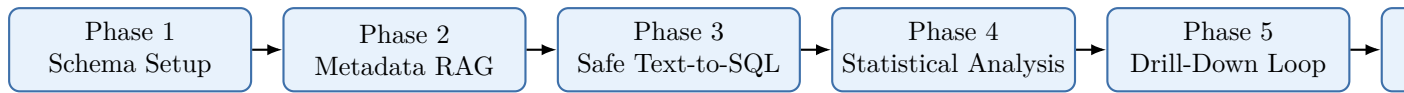


Figure 10: Incremental POC development roadmap.

13 Key Architectural Takeaway

Core Takeaway

**Think of the LLM as the analyst's reasoning brain,
and the database as the analyst's high-
speed calculator and factual ground truth.**